Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

Área Profesional Ingeniería

Introducción a la Programacion y Computacion

“Manual Tecnico Practica 01 IPC2 2026 1 semestre”

César Enrique Ixquiac Vásquez./ Carné 202230332

Quetzaltenango, 12 de Marzo 2025

“Id y enseñad a todos”

1. Introducción y Objetivos

Objetivo: Explicar el diseño arquitectónico, la estructura a nivel de código

(paquetes y clases), la gestión de concurrencia y el modelo relacional del sistema "Pizza Express Tycoon", proporcionando una guía completa para su mantenimiento, refactorización y escalabilidad.

Alcance del sistema: El sistema gestiona múltiples flujos de forma simultánea: simulación de partidas en tiempo real con generación de hilos asíncronos, administración aislada por sucursal (productos locales) y una administración global centralizada (usuarios, sucursales y parámetros). Implementa Control de Acceso Basado en Roles (RBAC) y persistencia de datos relacional mediante JDBC.

2. Arquitectura del Sistema y Patrones de Diseño

El proyecto está desarrollado bajo una arquitectura monolítica utilizando el patrón **DAO (Data Access Object)** en conjunto con una separación lógica inspirada en **MVC (Modelo-Vista-Controlador)**.

- **Aislamiento de la Interfaz:** Las Vistas (Swing) nunca ejecutan código SQL directamente.
- **Desacoplamiento:** Los Modelos (POJOs) transfieren los datos entre la base de datos y la interfaz gráfica.
- **Seguridad de Hilos (Thread Safety):** Uso de planificadores concurrentes para no bloquear el Hilo Despachador de Eventos (EDT) de Java Swing.

3. Estructura del Proyecto (Paquetes y Clases Principales)

El código fuente está organizado modularmente en los siguientes paquetes:

Paquete model (Entidades de Negocio):

Contiene las clases Planas de Java (POJOs) que mapean las tablas de la base de datos.

- Usuario.java: Gestiona credenciales, rol (id_rol) y sucursal asignada.
- Pedido.java: Almacena el detalle del producto, tiempos límite y el estado de preparación.
- Sucursal.java y Nivel.java: Entidades de configuración para la administración global.

Paquete dao (Data Access Objects):

Concentra toda la lógica de persistencia. Implementa sentencias preparadas (PreparedStatement) para evitar inyección SQL.

- SuperAdminDAO.java: Maneja consultas complejas (JOIN, SUM, GROUP BY) para estadísticas globales, rankings y el CRUD de todas las sucursales, usuarios y parámetros base.
- AdminTiendaDAO.java: Restringe las consultas mediante la cláusula WHERE id_sucursal = ? para garantizar el aislamiento de datos locales.
- PartidaDAO.java / PedidoDAO.java: Controlan el guardado del historial de estados durante el flujo del juego.

Paquete view (Interfaces Gráficas):

Desarrolladas con la API Java Swing.

- LoginFrame.java: Punto de entrada. Autentica y enruta mediante un switch(rol).
- GameFrame.java: Interfaz principal del Jugador. Contiene los contenedores dinámicos donde se renderizan los pedidos y el temporizador global.

- SuperAdminFrame.java: Implementa JTabbedPane para navegar entre los módulos de configuración global, renderizando datos dinámicos en componentes JTable.

Paquete **util** (Utilidades y Concurrencia):

- ConexionDB.java: Implementa el patrón *Singleton* para garantizar una única instancia de conexión a la base de datos MySQL, optimizando el uso de memoria.

- PedidoGenerator.java (o similar): Clase crítica que encapsula la lógica de los hilos de generación.

4. Lógica de Negocio y Componentes Críticos

4.1. Concurrencia y Generación de Pedidos (Multithreading)

Para evitar que la interfaz de usuario se congele, el "spawn" de pedidos no utiliza Thread.sleep(). En su lugar, se implementó un ScheduledExecutorService.

- **Funcionamiento:** Este pool de hilos planifica la ejecución de una tarea (Runnable) a intervalos fijos (ej. cada 5 o 10 segundos).

- **Validación:** Antes de lanzar un pedido, el hilo consulta la BD para obtener *únicamente* los productos con estado activo = 1 en la sucursal del jugador en turno.

4.2. Flujo de Estados del Pedido

Se maneja una máquina de estados finitos que valida transiciones estrictas en la base de datos:

- *Recibido* (1) ->*Preparando* (2) ->*Entregado* (4).
- *Cancelado* (5): Solo permitido si el estado actual es Recibido o Preparando (Aplica penalización).

- *Vencido* (6): Transición automática gestionada por el temporizador interno de cada pedido al llegar a cero.

4.3. Motor de Dificultad Dinámica

La clase encargada de la partida evalúa constantemente el puntaje o la cantidad de pedidos entregados. Al cruzar el umbral definido en la tabla nivel (ej. pedidos_para_subir), el sistema actualiza el id_nivel_alcanzado y extrae el nuevo tiempo_base_segundos, aumentando la dificultad en tiempo real.

5. Modelo Relacional de Base de Datos

La base de datos relacional está altamente normalizada para mantener la integridad referencial mediante llaves foráneas (FK):

- **usuario:** Llave foránea hacia rol y hacia sucursal (permite nulos para Super Admins).
- **sucursal:** Entidad central que agrupa productos locales, usuarios (empleados) y partidas.
- **producto_sucursal:** Tabla pivote que permite activar o desactivar productos independientemente en cada local.
- **partida / pedido:** Relación 1 a muchos. Almacenan todo el historial del jugador para futuras auditorías o cálculos estadísticos (Rankings).

6. Requisitos y Entorno de Despliegue

- **Lenguaje:** Java (JDK 11+)
- **Base de Datos:** MySQL Server 8.0+ / MariaDB 10+ (Motor InnoDB requerido para soporte de transacciones y FKs).
- **Librerías Externas:** mysql-connector-j.jar (JDBC Driver).
- **Hardware Mínimo:** CPU Dual Core, 4GB RAM, 500MB de almacenamiento.
- **Despliegue:** 1. Montar el script pizza_tycoon_db.sql en el puerto 3306.

  2. Configurar las credenciales en util/ConexionDB.java.

  3. Compilar el proyecto ejecutando el *Clean and Build* en NetBeans.

  4. Ejecutar el artefacto .jar resultante.